



Tipuri de date non-omogene

Tipul struct · Structuri de date din STL

Andrei Mihai Ciobanu

Algolymp Summer School

Tipul struct

Tipuri de date din STL

Recapitulare

Tipul struct

Problema: tablouri paralele

3 tablouri separate

	[0]	[1]	[2]
name []	Ion	Ana	Dan
	...		
	alte date		
	[0]	[1]	[2]
scor []	90	95	75
	...		
	alte date		
	[0]	[1]	[2]
varsta []	15	14	16

elev[0]: date în
3 zone de memorie!

VS

vector<Student>

	name	scor	varsta
v[0]	Ion	90	15
v[1]	Ana	95	14
v[2]	Dan	75	16

elev[0]: toate datele
compact, la un loc!

De ce struct?

Un singur sort sau swap mută **toate câmpurile** în bloc — nicio desincronizare posibilă.

Ce este un struct?

Multe probleme descriu **obiecte** cu mai multe proprietăți:

produs = *greutate* + *preț*

student = *nume* + *scor* + *vârstă*

eveniment = *timp* + *tip* + *id*

Un **struct** definește un **tip nou** grupând mai multe câmpuri:

```
1 struct produs {  
2     int greutate;  
3     double pret;  
4 };
```

De reținut

- produs devine un tip la fel ca `int` sau `string`
- Câmpurile pot fi de **orice tip**, inclusiv alte struct-uri

Declararea variabilelor struct

Odată definit, produs se folosește ca orice alt tip de date:

```
1 produs mar;           // o variabila de tip produs
2 produs banana, pepene; // mai multe variabile
```

Variabilele pot fi declarate și direct în definiție:

```
1 struct produs {
2     int greutate;
3     double pret;
4 } mar, banana, pepene;
```

Struct la nivel global

Declară struct-ul **înainte de** main(). Astfel e disponibil în toate funcțiile programului.

Struct Student — exemplu practic

Definim un tip Student cu trei câmpuri de tipuri diferite:

```
1 struct Student {  
2     string nume;  
3     int scor;  
4     int varsta;  
5 };
```

Tipul Student se poate folosi oriunde, ca orice alt tip:

```
1 Student elev;           // o singura variabila  
2 Student clasa[30];      // tablou de 30 de studenti  
3 vector<Student> v;      // vector de studenti
```

Un struct poate conține câmpuri de **orice tip**, inclusiv string, vector, sau alte struct-uri.

Accesul la câmpuri — operatorul .

Câmpurile se citesc și se modifică cu **operatorul punct (.)**:

```
1 Student s;  
2  
3 // setare campuri  
4 s.nume   = "Ion";  
5 s.scor   = 90;  
6 s.varsta = 15;  
7  
8 // citire campuri  
9 cout << s.nume;           // "Ion"  
10 cout << s.scor + 5;      // 95  
11 cout << s.varsta;        // 15
```

Fiecare câmp se comportă exact ca o variabilă de tipul său:

s.scor este int, s.nume este string.

Inițializare directă cu acolade

```
1 // ordinea campurilor: nume, scor, varsta (ca in struct)
2 Student s1 = {"Ana", 95, 14};
3 Student s2 = {"Ion", 90, 15};
4
5 // campuri la valoarea implicita (0 pentru numerice, ""
   // pentru string)
6 Student vid = {};
7
8 // mai multe variabile de acelasi tip
9 Student a, b, c;
10 a.scor = 10;
11 b.scor = 20;
12 c.scor = 30;
```

Atenție la ordine

{"Ana", 95, 14} corespunde câmpurilor în **exact ordinea din definiția struct.**

Tablou de struct-uri

Struct-urile se pot pune într-un **tablou**, ca orice alt tip:

```
1 Student cls[30];      // tablou de 30 de studenti
2 int n;
3 cin >> n;
4
5 for (int i = 0; i < n; i++) {
6     cin >> cls[i].nume >> cls[i].scor >> cls[i].varsta;
7 }
8
9 // accesul la campuri: cls[i].camp
10 cout << cls[0].nume;    // numele primului student
11 cout << cls[1].scor;    // scorul celui de-al doilea
```

Sintaxă

`cls[i].camp` — mai întâi indexul, apoi câmpul cu `.`

Struct în vector

Preferăm `vector<Student>` față de tabloul cu dimensiune fixă:

```
1 vector<Student> v;  
2 v.push_back({"Ion", 90, 15});  
3 v.push_back({"Ana", 95, 14});  
4 v.push_back({"Dan", 75, 16});  
5  
6 cout << v[0].nume;    // "Ion"  
7 cout << v[1].scor;    // 95  
8 cout << v.size();     // 3
```

Avantaj față de tablouri paralele

Un singur `push_back` adaugă toate câmpurile simultan. Nu mai există risc de desincronizare.

Struct și funcții

```
1 // by reference: eficient, modifica originalul
2 void mareste_scor(Student& s, int delta) {
3     s.scor += delta;
4 }
5
6 // returneaza un struct
7 Student citeste() {
8     Student s;
9     cin >> s.nume >> s.scor >> s.varsta;
10    return s;
11 }
```

Prin valoare vs. prin referință

`void f(Student s)` — copiază struct-ul (costisitor).

`void f(Student& s)` — lucrează pe original (preferat când vrei să modifice câmpurile).

Sortare cu comparator

Vrem: **scor** ↓, la egalitate **vârstă** ↑, la egalitate **nume** lexicografic.

```
1 bool cmp(const Student& a, const Student& b) {  
2     if (a.scor != b.scor) return a.scor > b.scor;  
3     if (a.varsta != b.varsta) return a.varsta < b.varsta;  
4     return a.nume < b.nume;  
5 }  
6  
7 sort(v.begin(), v.end(), cmp);
```

Regulă critică

Returnează false când elementele sunt egale.

Nu folosi `<=` sau `>=`: sort se comportă nedefinit (*strict weak ordering*).

Exemplu

Date de intrare:

Nume	Scor	Vârstă
ana	90	15
bogdan	90	14
cristi	100	17
daria	90	14

Rezultat sortat:

1. **cristi** — scor 100
2. **bogdan** — scor 90, vârstă 14
3. **daria** — scor 90, vârstă 14,
"daria" > "bogdan"
4. **ana** — scor 90, vârstă 15

Dacă ieșirea este greșită **doar** la valori egale, bug-ul este în comparator.

Tipuri de date din STL

Ce este STL?

STL (Standard Template Library) face parte din C++ și oferă:

- **Containere** — structuri de date gata de folosit (vector, set, map, ...)
- **Algoritmi** — sort, find, min_element, ...
- **Iteratori** — mod de a parcurge containerele, similari cu pointerii

Includere

Fiecare container are propriul header: `#include <vector>`,
`#include <set>`, etc.

`using namespace std;` — necesar pentru toate.

STL — Categoriile de containere

STL organizează containerele în **4 categorii**:

Categorie	Container principale
Secvențiale	<code>vector</code> , <code>deque</code>
Adaptoare	<code>stack</code> , <code>queue</code> , <code>priority_queue</code>
Asociative	<code>set</code> , <code>multiset</code> , <code>map</code> , <code>multimap</code>
Neordonate	<code>unordered_set</code> , <code>unordered_map</code>

Ce să alegi?

- Acces prin index sau sortare? → `vector`
- Valori unice sau sortate? → `set`
- Asociere cheie/valoare? → `map`
- Stivă sau coadă? → `stack` / `queue`

STL — Algoritmi uzuali

Algoritmii STL lucrează pe orice container prin **iteratori**. Incluzi în `<algorithm>`:

```
1  #include <algorithm>
2  vector<int> v = {5, 2, 8, 1, 9, 3};
3
4  sort(v.begin(), v.end());           // [1,2,3,5,8,9]
5  reverse(v.begin(), v.end());       // [9,8,5,3,2,1]
6
7  sort(v.begin(), v.end());
8  int mx = *max_element(v.begin(), v.end()); // 9
9
10 // cauta valoarea 5 (vectorul trebuie sortat)
11 bool ex = binary_search(v.begin(), v.end(), 5); // true
12 int nr = count(v.begin(), v.end(), 5);          // 1
```

Toți algoritmii primesc **iteratori de început și de final**:
`v.begin()` și `v.end()`.

1. vector — operații de bază

`vector<T>` funcționează ca un tablou, dar cu **lungime variabilă**:

```
1 #include <vector>
2 vector<int> v;
3
4 v.push_back(3);    // v = [3]
5 v.push_back(7);    // v = [3, 7]
6 v.push_back(1);    // v = [3, 7, 1]
7
8 cout << v[0];      // 3    (indezare de la 0)
9 cout << v.size();  // 3
10 v.pop_back();      // v = [3, 7]
11 cout << v.empty(); // 0 (false)
12 v.clear();         // v = []
```

`push_back`: $O(1)$ în medie. `v[i]`: $O(1)$.

1. vector — inițializare și parcurgere

```
1 // initializare cu n elemente = 0
2 vector<int> a(5, 0);    // [0, 0, 0, 0, 0]
3
4 // initializare cu valori
5 vector<int> b = {3, 1, 4, 1, 5};
6
7 // parcurgere cu indice
8 for (int i = 0; i < (int)b.size(); i++) {
9     cout << b[i] << " ";    // 3 1 4 1 5
10 }
11 // parcurgere for-each
12 for (int x : b) {
13     cout << x << " ";
14 }
15 // sortare
16 sort(b.begin(), b.end());    // b = [1, 1, 3, 4, 5]
```

2. pair — pereche de valori

`pair<T1, T2>` stochează două valori accesate cu `.first` și `.second`:

```
1 #include <utility>
2
3 pair<int, int> p = {3, 7};
4 cout << p.first;    // 3
5 cout << p.second;   // 7
6
7 // sortare automata: intai dupa first, la egalitate dupa
   second
8 vector<pair<int,int>> v = {{5,2}, {3,8}, {3,1}};
9 sort(v.begin(), v.end());
10 // v = [(3,1), (3,8), (5,2)]
```

Un `pair` e ca un struct cu doi câmpii anonimi. Util pentru a returna două valori dintr-o funcție sau pentru chei compuse în `map`.

3. string — șir de caractere

```
1  #include <string>
2
3  string s = "hello";
4  s.size();           // 5
5  s[1];               // 'e'
6  s += " world";      // s = "hello world"
7
8  // subsir de la pozitia 6, lungime 5
9  s.substr(6, 5);     // "world"
10
11 // prima aparitie; npos daca nu gaseste
12 s.find("world");     // 6
13
14 // comparare lexicografica cu ==, <, >
15 "abc" < "abd";      // true
```

string se comportă ca un `vector<char>`: are `push_back`, `size`, parcurgere cu `for`.

4. set — elemente unice, sortate

`set<T>`: elemente **unice**, menținute automat în **ordine crescătoare**. Toate operațiile: $O(\log n)$.

```
1  #include <set>
2  set<int> s;
3  s.insert(3);      // s = {3}
4  s.insert(1);      // s = {1, 3}  <- sortat automat
5  s.insert(3);      // ignorat!    s = {1, 3}
6
7  s.count(3);        // 1   (exista)
8  s.count(7);        // 0   (nu exista)
9  s.erase(3);        // s = {1}
10
11 for (int x : s) cout << x;    // in ordine crescatoare
```

4. set — căutare și multiset

```
1  set<int> s = {1, 3, 5, 9, 14};
2
3  // cel mai mic element >= k
4  auto it = s.lower_bound(5);
5  cout << *it;          // 5
6  // cel mai mic element > k
7  auto it2 = s.upper_bound(5);
8  cout << *it2;         // 9
9  // cel mai mare element
10 auto last = s.end(); --last;
11 cout << *last;        // 14
```

multiset — duplicate permise

```
multiset<int> ms;
```

```
ms.insert(3); ms.insert(3); → {3, 3}
```

```
ms.count(3) = 2
```


5. map — cheie $\rightarrow$ valoare

`map<K, V>` asociază chei unice cu valori. Cheile sunt sortate automat, $O(\log n)$ per operație.

```
1  #include <map>
2  map<string, int> freq;
3  freq["mere"] = 5;
4  freq["pere"] = 3;
5  freq["mere"]++;           // freq["mere"] = 6
6
7  cout << freq["mere"];    // 6
8
9  // verifica existenta fara a crea cheia
10 freq.count("prune");      // 0 (nu exista)
11 if (freq.count("mere"))
12     cout << freq["mere"]; // 6
13
14 freq.erase("pere");
```

5. map — parcurgere și multimap

```
1 map<string, int> freq;
2 freq["ana"] = 3;
3 freq["dan"] = 2;
4 freq["ion"] = 1;
5
6 // parcurgere in ordine alfabetica dupa cheie
7 for (auto& p : freq) {
8     cout << p.first << ": " << p.second << "\n";
9 }
10 // ana: 3
11 // dan: 2
12 // ion: 1
```

multimap — chei duplicate

Permite mai multe valori pentru aceeași cheie.

Rar folosit: `map<K, vector<V>` este de obicei mai flexibil.

6. unordered_set și unordered_map

Variante bazate pe **hashing**: $O(1)$ în medie, dar **nesortate**.

set / map

- Sortat automat
- $O(\log n)$ garantat
- lower_bound disponibil
- Sigur pe concursuri

unordered_*

- Nesortat
- $O(1)$ în medie
- Fără lower_bound
- Poate fi atacat cu date special construite

```
1 #include <unordered_set>
2 #include <unordered_map>
3 unordered_set<int> us = {3, 1, 4, 1, 5};
4 unordered_map<string, int> um;
5 um["a"] = 1;  um["b"] = 2;
```

7. stack, queue, deque

stack — LIFO

```
1 stack<int> st;  
2 st.push(3);  
3 st.push(7);  
4 st.top();    // 7  
5 st.pop();  
6 st.top();    // 3
```

queue — FIFO

```
1 queue<int> q;  
2 q.push(3);  
3 q.push(7);  
4 q.front();   // 3  
5 q.pop();  
6 q.front();   // 7
```

deque

```
1 deque<int> d;  
2 d.push_back(3);  
3 d.push_front(1);  
4 // [1, 3]  
5 d.pop_back();  
6 d.pop_front();
```

- stack: ultimul intrat = primul ieșit (stivă)
- queue: primul intrat = primul ieșit (coadă)
- deque: inserare/ștergere la ambele capete

priority_queue — coadă cu priorități

Max-heap implicit: `top()` returnează întotdeauna **maximul**.

```
1 #include <queue>
2 priority_queue<int> pq;
3 pq.push(3);
4 pq.push(7);
5 pq.push(1);
6 cout << pq.top();    // 7    (maximul)
7 pq.pop();
8 cout << pq.top();    // 3
```

Min-heap

`priority_queue<int, vector<int>, greater<int>> pq_min;`
`pq_min.top()` returnează **minimul**.

tuple și bitset

tuple — generalizare a lui pair

Poate stoca oricâte valori; accesate cu `get<i>`:

```
1 #include <tuple>
2 tuple<int, string, double> t = {1, "abc", 3.14};
3 get<0>(t);    // 1
4 get<1>(t);    // "abc"
5 get<2>(t);    // 3.14
```

bitset<N> — secvență fixă de *N* biți

```
1 #include <bitset>
2 bitset<8> b("10110101");
3 b.count();    // 5 (biti de 1)
4 b.set(0);     // seteaza bitul 0
5 b[3];        // acces bit
```

Recapitulare

1. **Distinct Numbers** (CSES #1621)
2. **Student Registry STL** (AlgoCode #4278)

Română

- **cplusplus.com** — tutorial structuri C++
- **pbinfo.ro**:
 - Structuri de date neomogene
 - Tipul struct
 - Operații cu structuri
 - Structuri imbricate
 - Structuri, tablouri și funcții
 - Standard Template Library (STL)

Engleză

- **CPH** — cap. 4: Data Structures
- **GeeksforGeeks**:
 - STL in C++
 - Structures in C++
- **USACO Guide**:
 - Intro to Data Structures (Bronze)
 - Intro to Sets & Maps (Bronze)
 - Sorted Sets (Gold)
 - Custom Comparators (Gold)
- **Programiz**:
 - C++ Structures
 - Structure & Function